АКОС
ИДЗ №3
Вариант 39. Регулируемый перекрёсток

Выполнил: студент БПИ252 Гришин Алексей Андреевич

Содержание

1 Модель перекрёстка **2**

2 Четыре реализации IPC **3**

3 Сборка, ввод и запуск **5**

4 Владение ресурсами и завершение **6**

5 Результаты работы и проверки **6**

6 Использование ИИ и проверенные изменения **8**

A Пользовательские промпты **8**

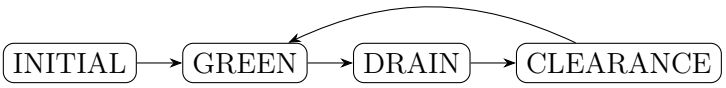
1 Модель перекрёстка

Вариант 39. Консольная программа на C17 моделирует движение автомобилей и пешеходов через общую конфликтную зону. Конфигурация задаёт направления, разрешённые манёвры, очереди, совместимость и набор фаз. Контроллер работает по фиксированной или адаптивной стратегии; экстренный запрос влияет на выбор фазы, сохраняя порядок FIFO. Для пешехода регистрируется запрос перехода.

При N направлениях работают координатор, контроллер, N генераторов и N семейств участников. На автомобиль не создаётся новый процесс: семейство обслуживает очередь своего направления. Максимум при $N = 16$ составляет 34 процесса. Модель и решения ролей общие для всех реализаций IPC.

Процесс	Поведение
Координатор	Проверяет запросы, фиксирует переходы, ведёт очереди и статистику, записывает предметные события в stdout и файл.
Контроллер	Получает снимок и выбирает фазу либо переход GREEN/DRAIN/CLEARANCE.
Генератор направления	По seed и номеру такта определяет прибытие и экстренность.
Семейство участников	Выбирает голову FIFO, запрашивает ENTER, ждёт срок проезда и передаёт LEAVE с активным ID.

1.1 Переходы и инварианты



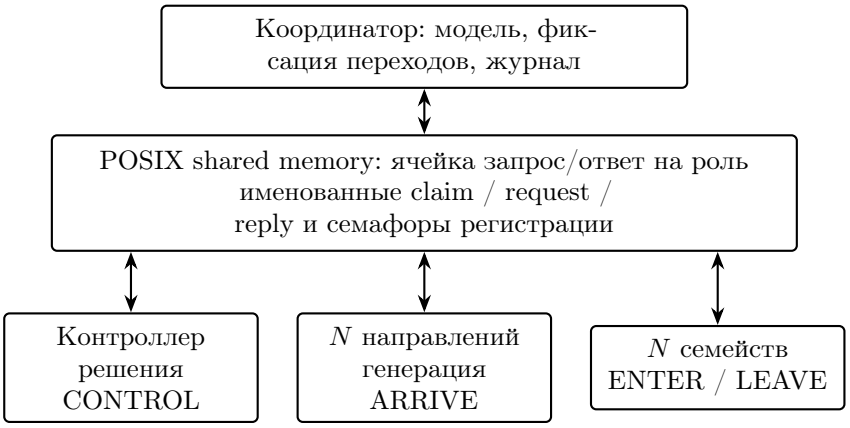
В GREEN разрешается въезд головы очереди при свободном направлении, совместимости с активными участниками и действующем окне допуска. Координатор проверяет условия и занимает зону одним переходом. DRAIN запрещает новые въезды, но сохраняет разрешение уже движущимся участникам. После их фактического выхода начинается полный CLEARANCE. Начатый проезд может закончиться позже окна `max_green`; следующая конфликтующая фаза всё равно ждёт освобождения зоны и защитного интервала.

Кольцевая очередь содержит до 64 ожидающих. Переполнение регистрируется как отказ, активный участник в очередь не входит. LEAVE принимается только для правильного ID и не раньше расчётного срока. Допуск проверяет $passed + active < target$, поэтому совместимые потоки не превышают цель. После первого STOP очереди и счётчики замораживаются.

Fixed выбирает фазы циклически. Adaptive сначала сравнивает возраст головы, затем длины очередей и циклический порядок. Приоритет может найти экстренный автомобиль в глубине FIFO, но он не обгоняет ожидающих перед ним участников. Минимальное зелёное время и clearance сохраняются.

2 Четыре реализации IPC

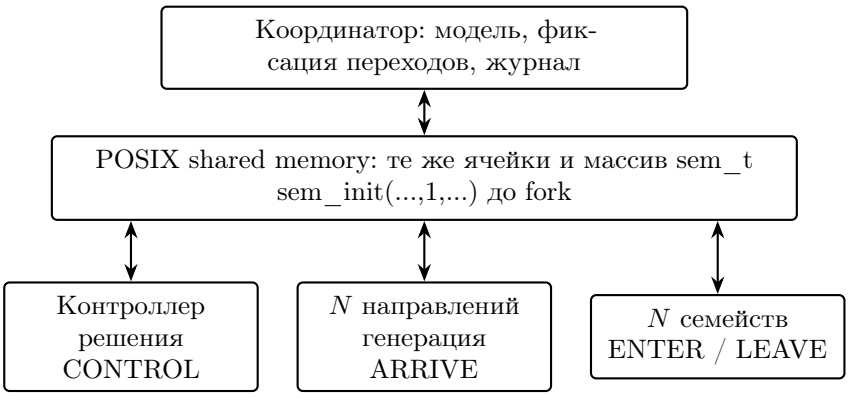
2.1 Fork, POSIX shm и именованные семафоры



`crossroad_named`: родитель создаёт $2N + 1$ детей через fork.
Родитель хранит PID, собирает детей и удаляет собственные имена IPC.

`shm_open` с O_CREAT|O_EXCL, `ftruncate` и `mmap(MAP_SHARED)` создают общую область. Семафоры создаются через `sem_open` с правами 0600. После записи кадра отправитель публикует request; координатор принимает запрос, записывает ответ и публикует reply. Одновременно у роли может быть только один незавершённый обмен.

2.2 Fork и process-shared неименованные семафоры



`crossroad_unnamed`: тот же цикл процессов и предметные правила.
sem_destroy выполняется родителем после завершения детей.

Семафоры находятся непосредственно в общем отображении; pshared равен 1. Инициализированные sem_t не копируются. При частичной ошибке уничтожается только успешно созданный префикс. Варианты named и unnamed сохраняются отдельными исполняемыми файлами.

2.3 Независимые программы и shared memory



`crossroad_shm`: координатор не создаёт рабочие процессы.
Контроллер, направления и семейства запускаются отдельными программами.

Каждая программа читает CONFIG. Короткая запись ready содержит версию, транспорт, SID, PID владельца, число ролей и контрольную сумму конфигурации. Она публикуется rename после полной инициализации. Рабочие могут ждать ready до запуска координатора. Claim закрепляет роль за одним процессом, семафор регистрации сериализует присоединение и закрытие сессии. Несовпадение конфигурации и дубликаты отклоняются.

2.4 Независимые программы и FIFO



`crossroad_fifo`: модельные данные передаются только через FIFO.
Открытие неблокирующее; чтение и запись ограничены монотонным сроком.

Рабочий открывает читатель ответа до записи первого запроса; координатор открывает писатель ответа при получении запроса. Это исключает взаимное ожидание открытий. Частичные кадры накапливаются; EINTR/EAGAIN повторяются с проверкой срока. EOF и EPIPE означают потерю связи.

Кадр содержит 128 явно заданных 64-разрядных слов, то есть 1024 байта. Проверяются magic, версия, SID, роль, операция, последовательный номер и диапазоны аргументов. В shared memory нет указателей, в FIFO не передаётся padding C-структур. Все процессы используют одну сборку Linux. Каждый write ограничен фактическим PIPE_BUF, полученным через fpathconf; один писатель на канал позволяет передать кадр несколькими записями.

3 Сборка, ввод и запуск

Нужны Linux, CMake 3.20+, компилятор C17 и Python 3.8+ для launcher и тестов. Применяются предупреждения -Wall -Wextra -Wpedantic -Wconversion -Wshadow. Все команды выполняются из корня проекта.

```
./tests/run.sh none
./tests/run.sh asan
mkdir -p results
./build/crossroad_named data/demo.conf results/named.log
./build/crossroad_unnamed data/compatible.conf results/unnamed.log
python3 scripts/run_session.py --transport shm --build-dir build data/minimal.conf
↪  results/shm.log
python3 scripts/run_session.py --transport fifo --build-dir build data/minimal.conf
↪  results/fifo.log
```

Сборка без тестов: CMake с `-DBUILD_TESTING=OFF`; Python для семи C-приложений не нужен. Для ASan/UBSan используется отдельный каталог `build-asan` и опция `ENABLE_SANITIZERS=ON`.

Интерфейс координатора:

```
crossroad_named|crossroad_unnamed|crossroad_shm|crossroad_fifo CONFIG [LOG] [--session
↪  DIR]
```

Журнал по умолчанию `crossroad.log`. Каталог сессии должен быть новым; без опции используется частный `.crossroad-PID`. Интерфейс ролей:

```
crossroad_controller CONFIG --session DIR --transport shm|fifo
crossroad_direction CONFIG --session DIR --transport shm|fifo --stream NAME
crossroad_participants CONFIG --session DIR --transport shm|fifo --stream NAME
```

Для minimal нужны controller, direction A/P и participants A/P. Каждая строка ручного запуска выполняется в отдельном терминале; координатор и роли используют один CONFIG. Launcher автоматизирует этот запуск и собирает своих детей, не участвуя в предметной модели.

3.1 Формат конфигурации

```
stream NAME KIND DIRECTION MANEUVER CAPACITY CROSSING_TICKS RATE
conflict NAME_A NAME_B VALUE
phase NAME STREAM_1 STREAM_2 ...
```

KIND: car или pedestrian. DIRECTION: north, east, south, west. Манёвры автомобиля: straight, right, left; пешехода: walk. RATE от 0 до 1000 задаёт вероятность попытки прибытия в тысячных долях на обработанный такт. Вместимость 1–64, проезд 1–1000 тактов и не больше max_green. Направлений и фаз от 1 до 16; каждое направление обслуживается, внутри фазы направления совместимы.

Параметр	По умолчанию	Диапазон
ticks / target	120 / 0	0–1000000000; ноль отключает предел
seed	39	0–4294967295
strategy	adaptive	fixed / adaptive
min_green / max_green	4 / 10	1–10000; min не больше max
clearance	2	1–10000 тактов
priority	1	0 / 1
emergency_rate	20	0–1000
tick_ms / delay_ms	1 / 0	1–60000 / 0–60000 мс

Период в наносекундах проверяется на переполнение. Имена имеют 1–31 ASCII-букву, цифру, подчёркивание или дефис. Повторные имена запрещены. Числа десятичные, без знаков и суффиксов. Допускаются комментарии после #, CRLF и последняя строка без newline; NUL и файл длиннее 16384 байт отклоняются. Повтор скалярного параметра заменяет значение. Полный пример есть в README.

4 Владение ресурсами и завершение

Общая область содержит сообщения и синхронизацию; очереди и статистика локальны у координатора. Рабочие принимают решения по снимкам. Время CLOCK_MONOTONIC начинается после HELLO всех ролей. SplitMix64 адресуется по seed, направлению и такту; пропуски тактов не создают запоздалых прибытий. delay_ms регулирует только частоту сообщений состояния.

Координатор держит единственный дескриптор журнала. Перед его усечением сравниваются inode и устройство реально открытых входного файла и журнала. Вывод выполняется через open/read/write/close, с обработкой коротких операций. При отказе канала исправный продолжает сохранять события.

Период, точная цель, SIGINT/SIGTERM, ошибка и недостижимая цель завершают модель. Первый STOP фиксирует причину и статистику. Ожидающие остаются в очередях, незавершённый проезд остаётся в зоне; BYE не засчитывается как выход.

$$\text{прибыло} = \text{прошло} + \text{отказы} + \text{очереди} + \text{в зоне}$$

На startup отводится 5 с, на отсутствие обмена после старта 5 с, на STOP ещё 5 с целиком. Обработчик сигнала только устанавливает sig_atomic_t; обычный код выполняет STOP и уборку. Родитель сохраняет успешные PID, проверяет статусы и собирает детей через waitpid. Независимый координатор не собирает чужие процессы. Рабочие ограниченно выходят при потере связи.

Владелец удаляет только имена своей сессии. Флаги создания и счётчик инициализаций обеспечивают откат частично созданного набора. Семафоры unnamed уничтожаются после завершения детей, затем снимается отображение. Коды: 0 штатно, 1 ошибка, 2 неверный ввод, 3 недостижимая цель, 128+signal прерывание.

5 Результаты работы и проверки

Проверки выполнены в Debian 12/aarch64, GCC 12.2.0, CMake 3.25.1, Python 3.11.2. Исходники копировались в отдельный каталог Linux без старого проекта и готовых бинарников. Сборки Debug и ASan/UBSan выполнялись командами `./tests/run.sh none` и `./tests/run.sh asan`.

Проверка	Обычная	ASan/UBSan
Модельные границы и конфигурация	пройдено	пройдено
Сценарии четырёх backend	92/92	92/92
Ручной независимый запуск	2/2	2/2
Регистрация и неполный набор ролей	2/2	2/2
CTest	2/2	2/2
Время CTest	41.31 с	103.39 с

Проверены demo/hand/emergency/compatible/minimal, обе стратегии, период и цель, крайние seed, максимальные 16 направлений с вместимостью 64, две одновременные сессии, сигналы координатору и рабочему, /dev/full, закрытый и заполненный stdout. Рабочие запускались до координатора. Проверены дубликат роли, несовпадающий CONFIG и отсутствие части ролей. После завершения проверяются PID и отсутствие имён shm/sem своей сессии. Ошибочные конфигурации отклоняются до IPC; попытка использовать входной файл как журнал не меняет его содержимое.

Независимый Python-проверяющий восстанавливает FIFO и активные проезды по журналу, проверяет разрешённые фазы, конфликты, длительность проезда, clearance и баланс. Повреждённый итоговый счётчик отвергается. В C-проверке используется подставное время: ранний LEAVE, неверный ID, DRAIN с активным участником, clearance за 1 нс до границы и на границе, заморозка после STOP. В sanitizer-сборке диагностики ASan/UBSan не получено.

5.1 Примеры четырёх реализаций

Следующие Linux-запуски использовали data/minimal.conf: два направления, fixed, seed 39, период 100 тактов по 20 мс. В каждой реализации работали шесть процессов модели. Асинхронность допускает разные числа прибытий и проездов при одинаковом seed; проверяется семантика журнала.

Backend	Прибыло	Прошло	Отказ	Очередь	Зона	Ср. ожидание
named	83	47	33	2	1	4.53
unnamed	83	48	32	2	1	4.77
shm	83	49	33	1	0	4.78
fifo	83	46	34	2	1	4.46

named

```
[t=0] Начало фазы: имя=cars поколение=1 событие=7 время_нс=5923167
[t=0] Разрешение: поток=A id=2 очередь=0 срок=34035084 событие=9 время_нс=14035084
[t=100] Завершение: причина=1 сигнал=0 код=0 событие=469 время_нс=2000000000
[t=100] ИТОГО: прибыло=83 прошло=47 отказов=33 ожидают=2 в_зоне=1 ожидание=213 среднее=4.53
↪  максимум=9 пик=2 процессов=6 событие=472 время_нс=2000000000
```

unnamed

```
[t=0] Начало фазы: имя=cars поколение=1 событие=2 время_нс=4000375
[t=0] Разрешение: поток=A id=1 очередь=0 срок=30011875 событие=9 время_нс=10011875
[t=100] Завершение: причина=1 сигнал=0 код=0 событие=470 время_нс=2000000000
[t=100] ИТОГО: прибыло=83 прошло=48 отказов=32 ожидают=2 в_зоне=1 ожидание=229 среднее=4.77
↪  максимум=10 пик=2 процессов=6 событие=473 время_нс=2000000000
```

shm

```
[t=0] Начало фазы: имя=cars поколение=1 событие=5 время_нс=4165834
[t=0] Разрешение: поток=A id=2 очередь=0 срок=30210834 событие=9 время_нс=10210834
[t=100] Завершение: причина=1 сигнал=0 код=0 событие=465 время_нс=2000000000
[t=100] ИТОГО: прибыло=83 прошло=49 отказов=33 ожидают=1 в_зоне=0 ожидание=234 среднее=4.78
↪  максимум=10 пик=2 процессов=6 событие=468 время_нс=2000000000
```

fifo

```
[t=0] Начало фазы: имя=cars поколение=1 событие=2 время_нс=5962875
[t=0] Разрешение: поток=A id=1 очередь=0 срок=32046500 событие=9 время_нс=12046500
[t=100] Завершение: причина=1 сигнал=0 код=0 событие=466 время_нс=2000000000
[t=100] ИТОГО: прибыло=83 прошло=46 отказов=34 ожидают=2 в_зоне=1 ожидание=205 среднее=4.46
↪  максимум=9 пик=2 процессов=6 событие=469 время_нс=2000000000
```

6 Использование ИИ и проверенные изменения

ИИ-ассистент (LLM) использовался при анализе условий, составлении плана, реализации потоков и синхронизации, подготовке проверяющего скрипта, сборочных сценариев и документации. Первичный код, предложенный нейросетью, содержал избыточные абстракции. Я провел глубокий рефакторинг, в результате которого оставил только необходимую и надежную логику.

A Пользовательские промпты

Промпт 1

> Рассмотри въезд на перекрёсток как конкурентный протокол. Контроллер уже закрыл фазу, но семейство участников ещё видит старый снимок с зелёным сигналом. Построй минимальную последовательность сообщений, приводящую к ошибочному въезду. Определи точку фиксации ENTER и проверки, которые координатор обязан повторить при получении запроса. Отдельно разбери одновременный допуск двух совместимых направлений, когда до цели остался один участник. Предложи проверку, различающую безопасную и ошибочную реализации.

Промпт 2

> Автомобиль въехал в последний момент зелёной фазы, а длительность его проезда равна max_green. Покажи временную диаграмму GREEN, DRAIN и CLEARANCE. От какого события отсчитывать защитный интервал, если расчётное время проезда истекло, но LEAVE ещё не зарегистрирован? Разбери границы B-1, B и B+1 наносекунда. Объясни, какие события допустимы, если одновременно наступают конец периода, достижение цели и сигнал остановки, и какие счётчики после STOP должны остаться неизменными.

Промпт 3

> Спроектируй две версии обмена через POSIX shared memory: с именованными семафорами и с sem_init(..., 1, ...). Для каждой укажи владельца отображения, имён и семафоров, начальные значения и порядок публикации запроса и ответа. Составь таблицу отката после ошибки на каждом шаге создания. Объясни, почему обычная память после fork не заменяет shared memory, почему нельзя копировать инициализированный sem_t и когда безопасно вызывать sem_destroy. Общий цикл обмена должен оставаться одинаковым в обеих версиях.

Промпт 4

Координатор и рабочие запускаются из отдельных терминалов в произвольном порядке. Предложи протокол регистрации, который различает незавершённое создание сессии, несовпадающую конфигурацию и уже занятую роль. Разбери гонку между присоединением рабочего и началом STOP. Укажи, какие ожидания должны иметь общий абсолютный срок. Предложи сценарии с отсутствующей ролью, двумя одновременными сессиями и смертью координатора. Не используй fork в независимом координаторе и не собирай чужие процессы через waitpid.

Промпт 5

Нужно передать фиксированный кадр 1024 байта через два FIFO на роль, используя именованные семафоры. Предложи порядок открытия концов без взаимной блокировки и скрытого обмена через shm. Рассмотри PIPE_BUF, равный 512 и 4096 байтам, короткие операции, EINTR, EAGAIN, EOF и EPIPE. Покажи, как сохранять частично прочитанный кадр между вызовами и проверять срок даже при непрерывном EINTR. Объясни, какие гарантии даёт единственный писатель и какие поля кадра всё равно нужно проверять.

Промпт 6

Разработай независимую проверку по текстовому журналу, не вызывающую функции C-модели. Какие поля нужны, чтобы восстановить FIFO, разрешения, активные проезды и итоговый баланс? Придумай повреждения журнала, которые обязаны быть отвергнуты, включая ранний LEAVE и сокращённый clearance. Затем сформулируй проверки SplitMix64 при перестановке обращений и пропуске тактов. Раздели воспроизводимость случайной выборки и зависимость событий от планировщика: какие результаты двух запусков с одним seed можно сравнивать?